Wan 2.1 on an Apple M5: text-to-video in 8.6 GB, bit-identical to stock

Project report · Wan 2.1 T2V 1.3B via mlx-video · 832×480, 33 frames requested (36 output), 50 steps · measured on a MacBook Air M5 (16 GB) · 21 October 2026

Executive summary

Stock mlx-video cannot run two of Wan's three stages on a 16 GB Mac without heavy swapping. The composed pipeline runs the whole clip at 8.57 GB peak with swap flat, in 21.1 min, and its final latents are bit-identical to stock's. No step-caching fast mode ships: no configuration passed the quality gate on all three test prompts.

Stage	Stock mlx-video on this machine	Composed exact pipeline	Exactness vs stock
Text encoder (umT5-XXL, upcast to fp32)	**23.12 GB** MLX / 23.90 GB footprint, swap +5.6 GB, 141 s* (exp-101): **cannot run in 16 GB without heavy swap**	**2.07 GB** / 2.59 GB, 6.2 s, swap flat (exp-102)	**bit-identical** embeddings, prompt and negative prompt (re-checked inside the pipeline: exp-128 identical)
DiT denoise (30 blocks, 2.64 GB bf16)	4.64 GB MLX, 22.9 s/step (exp-130)	**2.42 GB**, +0.1–0.7% vs per-block-compiled resident (exp-126)	**bit-identical at 50 steps** (exp-127 = exp-130: identical)
VAE decode (3D causal, fp32)	tiled: **17.90–18.63 GB** MLX, swaps, 83–113 s* (exp-103, exp-132). Untiled: **cannot run** (one 11.04 GB buffer > the 9.53 GB Metal limit, exp-108)	**8.42 GB** / 8.19 GB, 56 s, swap flat (exp-125)	frames 1–16 **bit-identical** to stock untiled (the longest prefix stock can decode); all 36 vs stock tiled: 46.8 dB min / 56.0 dB mean
End to end	2 of 3 stages cannot run within 16 GB. With the encoder skipped (frozen embeddings) the rest still peaks at 18.07 GB with swap (exp-103)	**8.57 GB** footprint peak, **21.1 min**, swap flat (exp-127)	final latents **bit-identical**; video vs stock's default tiled decode of the same latents: 49.5 dB min / 58.7 dB mean, max pixel diff 17 (exp-132)

* Swap-inflated: valid for memory, not for speed (protocol rule R3). These are what stock takes on this machine, not stock's speed.

Two modes, one shipped

Mode	Time per clip	Peak	Claim
Exact (shipped)	20.4–21.1 min on three prompts	8.52–8.57 GB, swap flat	final latents bit-identical to stock mlx-video; frames identical to stock's exact decode wherever stock can run it
Fast (step caching): **not shipped**	12.5–16.7 min	8.61–8.70 GB	no configuration passed LPIPS < 0.05 on all three prompts (section 5); curve and reasons recorded

In plain terms. Stock software needs more memory than this laptop has for two of the three steps of making a video, so it either crawls while the disk stands in for memory, or crashes. We rebuilt how those steps use memory, without changing any arithmetic. The video now fits in about half the machine, and the numbers inside the model match the stock software's exactly. We also tried to make it faster by skipping repeated work; that worked on a calm scene but visibly changed busy scenes (waves, a rainy street), so it is not offered.

1. Where the memory goes

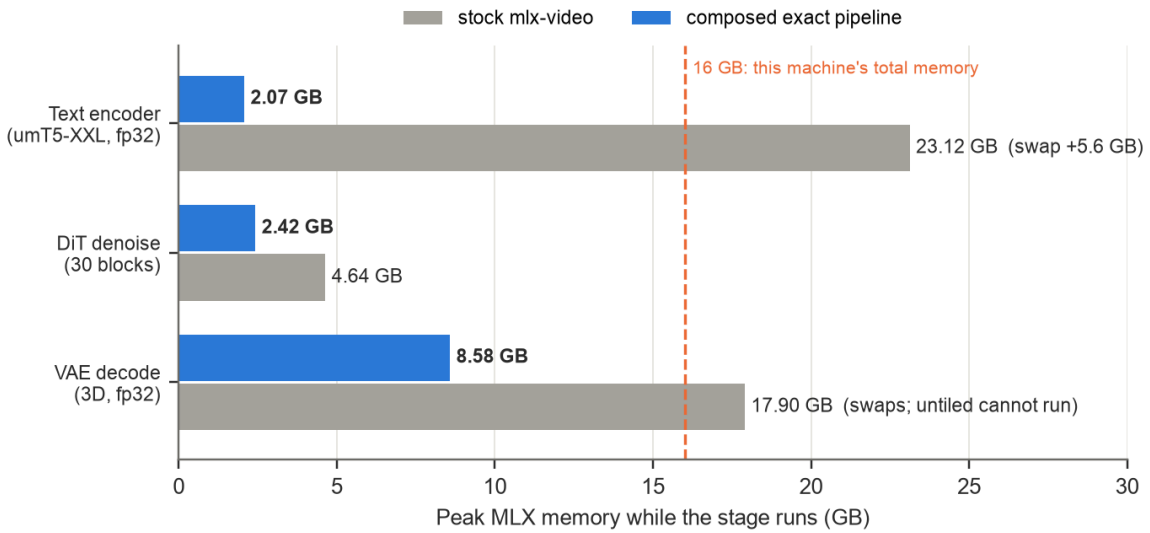


Figure 1. Peak MLX memory per stage, stock vs the composed exact pipeline. Stock encoder: exp-101; stock DiT: exp-130; stock decoder: exp-132 (tiled; untiled cannot run). Composed: exp-127.

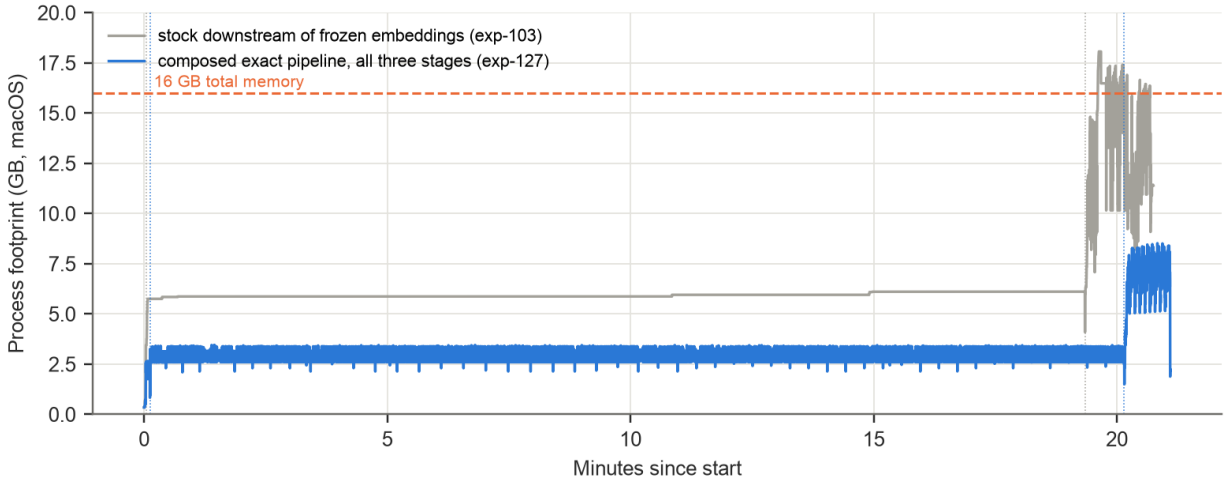


Figure 2. Process footprint over one full clip. Grey: stock mlx-video downstream of frozen stock embeddings (exp-103; the stock encoder alone needs 23.9 GB, exp-101, so it was run once separately). Blue: the composed pipeline, all three stages (exp-127). Dotted lines: start of denoising and of decoding. The grey spike at the end is the stock tiled decode.

Precisely. Encoder: mlx-video upcasts the 24-block umT5-XXL and its 256,384-row embedding table to fp32, 23.12 GB. DiT: 2.64 GB of bf16 weights plus activations for ~14,000 tokens at batch 2 (CFG). Decoder: each 3×3×3 causal convolution at full resolution needs a working buffer about 7.6× its input; one such convolution sets the stock peak (diagnosed with a per-module profile, exp-107).

2. How each stage was made to fit, exactly

Text encoder: stream the blocks, upcast as they load

Precisely. The 24 blocks are stored as bf16 files and streamed through two-block residency with prefetch; each is upcast to fp32 as it loads, so the arithmetic is the stock fp32 arithmetic. The embedding table is never loaded: only the rows for the prompt's tokens are read from disk. 23.12 → 2.07 GB, bit-identical (exp-101/102). A new variant of the frozen streaming pattern (upcast-on-load).

DiT: block weight streaming (frozen pattern, adapter only)

Precisely. The frozen engine streams the 30 blocks with prefetch and per-block compilation. mlx-video calls blocks with a positional argument the frozen engine does not accept, so a small adapter subclass folds it into a keyword; the engine is unchanged. 4.62 → 2.41 GB at +0.1–0.7% per step (two alternating A/B rounds, exp-126), below Klein's +2.1–2.8% as predicted (smaller blocks, more compute per block). Bit-identical to stock at 50 steps.

Decoder: a new exact 3D streaming decoder

Precisely. The frozen 2D chunked decoder did not apply (3D causal convolutions carry state across frames), and the frozen eval-barrier pattern had no effect (the peak is one operation's working buffer, not intermediates kept alive). The new decoder streams one latent frame at a time, keeping each causal convolution's last two input frames as state (zeros initially, identical to stock's causal padding), and builds each convolution window and each upsampled strip directly in horizontal strips. Strip heights are multiples of 24 rows and the input is padded once, so every strip sits on the same Winograd tile grid as the full convolution: re-padded strips shifted the grid and broke bit-identity (isolated test, exp-114).

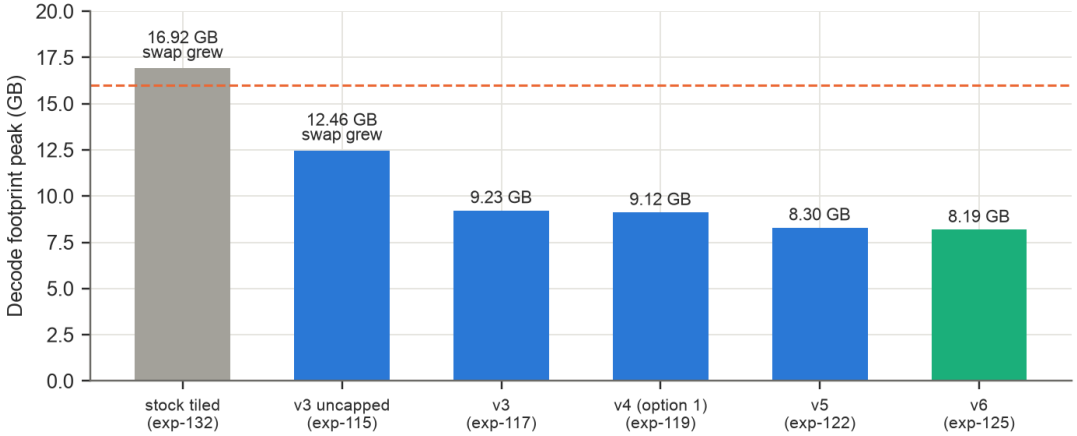


Figure 3. Decoder footprint by version (cache cap 0.5 GB from v3 on). v4 builds each convolution window directly (option 1); v5 builds upsampled strips directly; v6 adds evaluation barriers inside each chunk. Stock tiled for comparison (exp-132).

What is left in v6 is mostly the causal state itself (~3.8 GB of fp32 history across ~14 convolutions). The projection that option 1 would reach ~6 GB was wrong by about 2 GB because the next-largest component sat right behind the one it fixed; recorded as a library rule (memory fixes are serial: project against the sorted component list). Shrinking the state exactly needs spatial halves (deferred until an 8 GB target is decided).

3. Exactness, verified at the shipping configuration

Every check below was re-run by the report builder from the saved tensors (byte comparison), not quoted from logs.

Check	Runs	Result
Text embeddings used inside the pipeline vs the frozen stock capture	exp-128 vs frozen v1	**identical**
Initial noise vs a replay of stock mlx-video's random-number order	exp-129	**identical**
Final latents, streamed DiT vs stock DiT, 50 steps	exp-127 vs exp-130	**identical**
Final latents, streamed DiT on the frozen reference's random path, 50 steps	exp-131 vs frozen v1	**identical**
Decoded frames 1–16, streaming decoder vs stock untiled decode	exp-125 vs exp-109	**identical**
All 36 frames vs stock's default tiled decode of the true-stock latents	exp-127 vs exp-132	49.5 dB min / 58.7 dB mean (Tier E: > 40 dB)

A reference retired, and why

The first composed run disagreed completely with the frozen stock latents (cosine 0.06): a valid video, but a different one. Its embeddings and its starting noise both matched stock exactly, so the difference had to be upstream of denoising. Cause: the frozen reference had been made with a stub text-encoder loader that skips constructing the encoder, and constructing a model draws random numbers for its initial weights, so the reference's starting noise came from a shifted point of the seed-42 stream. Running the streamed DiT on that same stub path reproduces the old reference bit for bit (exp-131), confirming the cause; the true-stock reference is now exp-130. Rule recorded: a reference must be produced by the same random-number path as the system under test, and exactness is claimed at the step count that ships.



Figure 4. Fox prompt, frames 1, 18, 36. Left: stock mlx-video's own (tiled) decode of the true-stock latents (exp-132). Middle: the composed exact pipeline (exp-127); its latents are identical to stock's and its decoder is exact, so the visible difference to the left is stock's tiling (49.5 dB min). Right: the A2 fast mode, which passes on this prompt.

4. Speed: step caching (track closed, not shipped)

Precisely. A probe of the 50-step run (exp-104) found heavy repetition: 96% of block-steps change their output by less than 2%, and the with-prompt and without-prompt passes converge late. Near-duplicate tokens were rare (0.5% after centring), so token merging was dropped. One combined mechanism was built: on a skipped step all 30 blocks are bypassed and their last contribution reused; on a cond-only step only the with-prompt half runs. A2 is a fixed schedule from the probe (1.56× ideal); A3-d skips only when the measured change between full steps is below a threshold. The quality gate: LPIPS < 0.05 on every frame of three prompts, against each prompt's exact video; the dynamic criterion was pre-registered before its results existed.

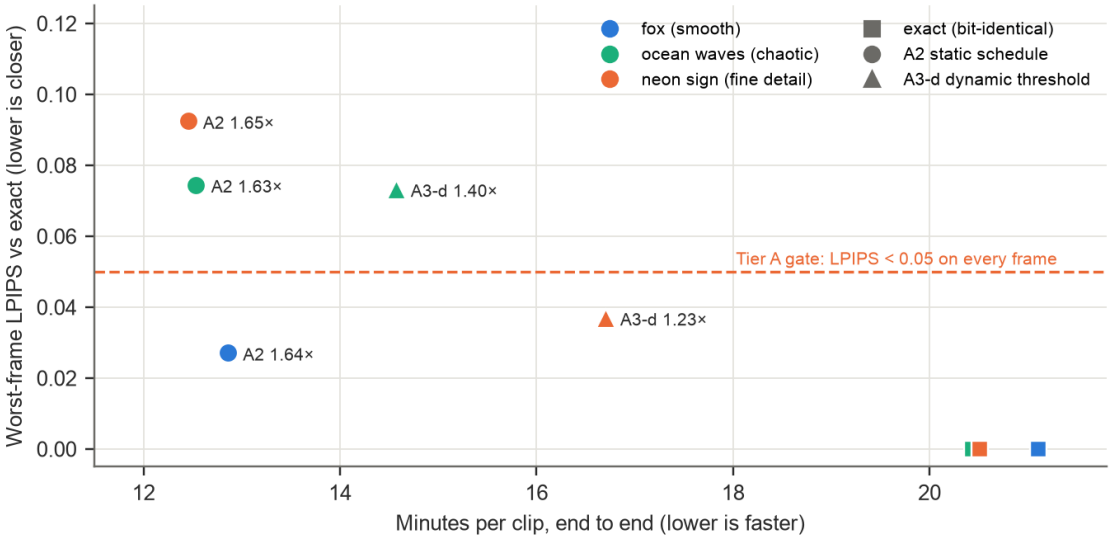


Figure 5. Speed vs worst-frame quality for every run. Squares: exact (LPIPS 0). Points below the dashed line pass.

Prompt	Exact	A2 (static)	A3-d (dynamic)
fox (smooth)	21.1 min†	12.9 min (1.64×), LPIPS max 0.027 ✓ pass; 0/36 frames > 0.05; motion 0.97×	not run (gate already failed on waves)
ocean waves (chaotic)	20.4 min	12.5 min (1.63×), LPIPS max 0.074 ✗ fail; 16/36 frames > 0.05; motion 0.98×	14.6 min (1.40×), LPIPS max 0.073 ✗ fail; 15/36 frames > 0.05; motion 0.98×
neon sign (fine detail)	20.5 min	12.5 min (1.65×), LPIPS max 0.092 ✗ fail; 36/36 frames > 0.05; motion 1.00×	16.7 min (1.23×), LPIPS max 0.037 ✓ pass; 0/36 frames > 0.05; motion 1.00×

† The fox's exact run had a fan added partway through; its fan-on equivalent is ~20.2 min. All cached runs: 8.61–8.70 GB, swap flat. Thermal note: skipped steps let this fanless chip rest, so A2's full steps ran at 22.2 s vs 23.2 s uncached; ~5% of A2's speed-up is that machine-specific effect.

Why no fast mode ships

No configuration passes all three prompts, and the failures are complementary: A2 passes only the fox, A3-d passes only neon, nothing passes waves. The one detail-safe point (A3-d on neon, 1.23×) is below the 1.3× bar set before the sweep. A further schedule (the window neon happened to choose, applied to every prompt) was declined: a schedule chosen by looking at which one passed is what pre-registration exists to block.

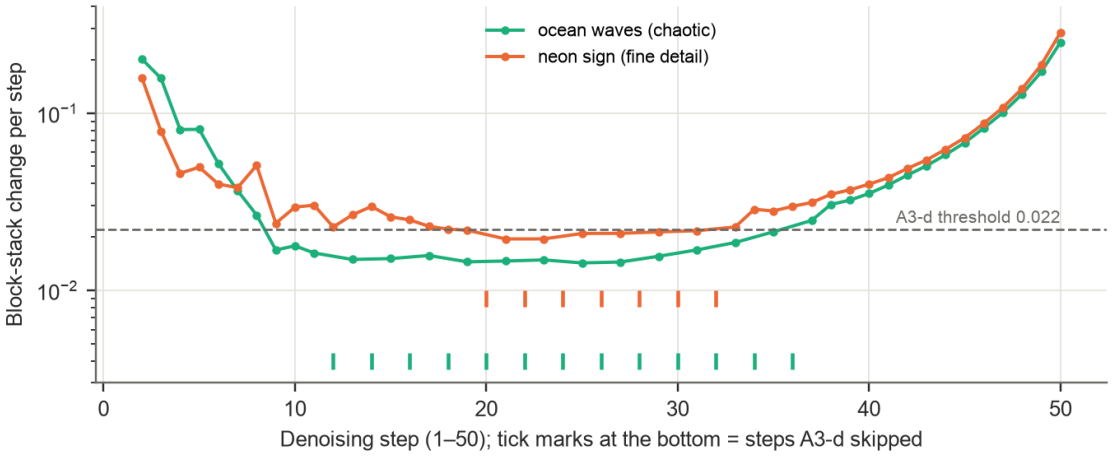


Figure 6. How much the block stack's output changes per denoising step (log scale), measured during the A3-d runs. Fast settling, a calm middle, then a sharp late rise as fine detail is painted in. Waves measures calmer than neon, so the threshold skipped it more and earlier, yet waves is the more fragile prompt.

Three findings. (1) *Method:* testing on the smooth fox alone would have shipped A2 at ~1.6× with a defect on chaotic and fine-detail content; the three-prompt gate caught it in one afternoon. (2) *Mechanism:* the cheap change signal measures how much changes, not whether it matters to the eye, and ranks prompts backwards on the fragile ones (small changes move foam; neon's larger changes land in stable structure). A future signal must be perceptual or content-weighted, or calibrated per prompt, and validated on a held-out fourth prompt. (3) *Assets kept:* the U-shaped change curve (reuse belongs in the calm middle; the late rise is untouchable), the three-mode controller, and a measured-absent interaction with weight streaming (block-0 stall after a skip ≤ 0.02 ms over 16 skips).



Figure 7. Neon sign, frames 1, 18, 36. A2 changes small background content (an extra yellow umbrella, moved pedestrians); A3-d stays close to exact. The garbled sign text is the 1.3B model's, present in the exact video too.

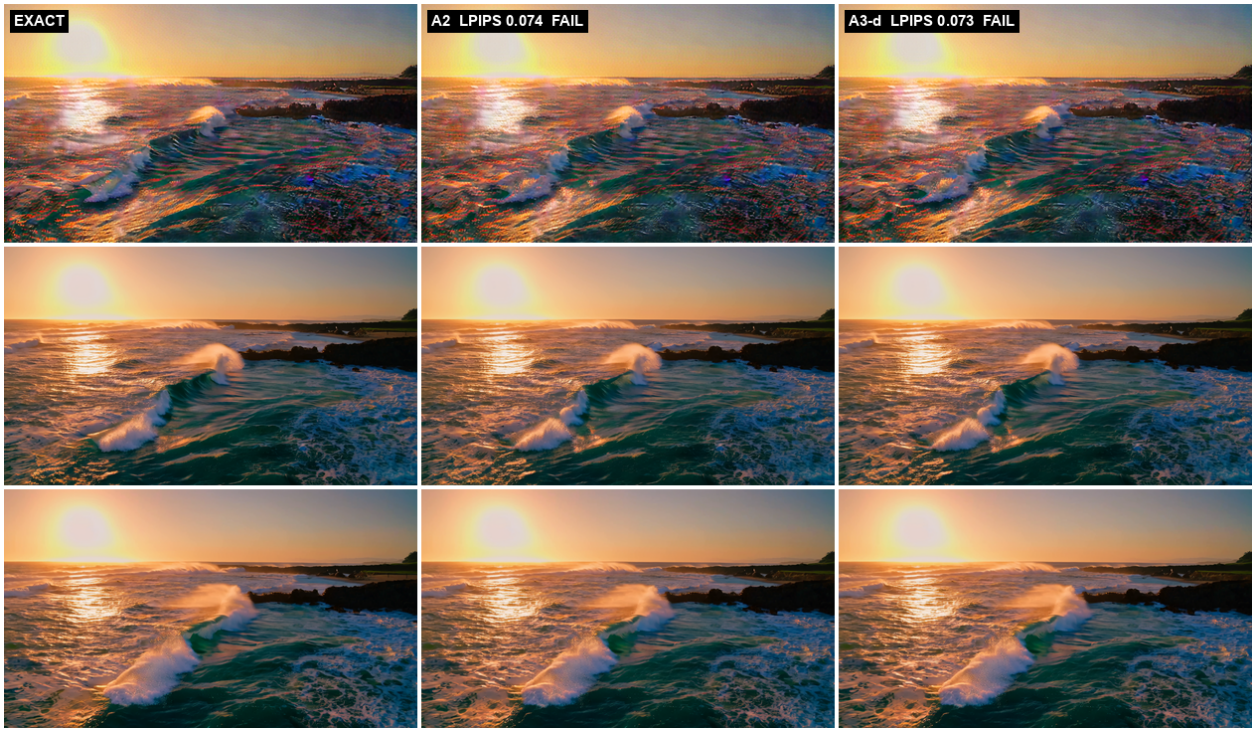


Figure 8. Ocean waves, frames 1, 18, 36. Same scene in all three; the shape and position of foam and spray differ. Motion is not slowed (0.98× exact). Failing the gate does not mean the video looks bad: it means 'indistinguishable from exact' cannot be claimed.

Private use is a different question from shipping: for smooth, stable-background content reviewed by eye before use, the A2 schedule (~13 instead of ~20.5 minutes) is a reasonable personal tool. It is not part of any claim in this report.

5. Did the image playbook transfer?

The pattern library and engine built for FLUX.2 Klein 4B (image) were frozen before Wan was touched; Wan work used adapters only. Klein cells come from the library's recorded results; '—' means not used on that model.

Pattern	Klein 4B (image)	Wan 2.1 1.3B (video)
Block weight streaming + prefetch	hit: 8.57 → 2.81 GB, +2.1–2.8%	**hit**: 4.62 → 2.41 GB, bit-identical to stock at 50 steps
Upcast-on-load streaming	—	**new variant**: 23.12 → 2.07 GB, bit-identical
Row-gathered embeddings	—	**hit**: the 256k-row table is never loaded
Allocator cache cap	hit: 11.54 → 10.26 GB	**hit**: 0.5 GB cap on every composed run
Free each stage after last use	hit: 15.99 → 9.51 GB	not needed: mlx-video already frees between stages
Evaluation barriers between blocks	hit: VAE 9.55 → 7.26 GB	**applies, no effect** on the decoder (one op's buffer, not graph lifetime)
Exact 2D row-chunked decode	hit: 1.81 GB, Tier E	**does not transfer** (3D causal) → new exact 3D streaming decoder
Kernel-grid-aligned chunking	—	**new**: Winograd-aligned strips keep chunked convolutions bit-identical
Step caching	not viable (4-step model)	**closed, not shipped** (section 4)
Token merging	—	**miss**: 0.5% near-duplicate tokens

Klein's memory work took two days; Wan's took one working day, from the encoder fix to the before/after table, on a model family the playbook had never seen, including one genuinely new component (the 3D decoder) and two live debugging detours.

Lessons recorded in the library

Rule	Evidence
Memory fixes are serial: project a fix against the sorted list of memory components, not its local saving	Klein exp-005/006; Wan decoder option 1 (projected ~6 GB, measured 8.2 GB)
Exactness is verified at the shipping step count, and a reference must share the system's random-number path	exp-126 (8 steps) vs exp-130/131 (50 steps); the retired reference
Denoising change is U-shaped: reuse belongs in the calm middle, never in the late rise	exp-a2-v3, exp-a3d-v2/v3 change logs
Chunked convolutions stay exact only on the kernel's tile grid	exp-114
On passively cooled hardware, skipped work also speeds the remaining work; report the schedule's share separately	exp-134 vs exp-131

6. Setup and experiment index

Machine	MacBook Air Mac17,3 (fanless; a fan was placed under it from ~12:30 on the day), Apple M5, 8-core GPU, 16 GB unified memory
Software	MLX 0.32.4; mlx-video @ 87db56a (MIT); LPIPS (AlexNet) via torch
Model	Wan-AI/Wan2.1-T2V-1.3B (Apache-2.0); 832×480, 33 frames requested (mlx-video outputs 36), 50 steps, UniPC, CFG 5, seed 42
Prompts	v1 a red fox trotting through snow; v2 ocean waves on a rocky beach at sunset; v3 a neon 'NOODLES' sign on a rainy street
R1 execution mode	bit-identity is only claimed against a reference produced in the same execution mode
R2 simulation	mx.set_memory_limit is a soft limit; no smaller-machine claim is made from it (a real 8 GB Mac is untested)
R3 timing validity	runs where swap grew are memory-only; cool-down to Nominal thermal state before every timed run
Tiers	E: bit-identical or PSNR > 40 dB. A: LPIPS < 0.05 on every frame (plus visual review)

Experiment	What it is	Footprint peak	Time	Swap
exp-101-te-stock	stock text encoder, once (frozen embeddings capture)	23.90 GB	141 s	grew
exp-102-te-stream	streamed text encoder	2.59 GB	6 s	flat
exp-103-base-auto	stock DiT + stock tiled decode on frozen embeddings	18.07 GB	1245 s	grew
exp-125-v6-full	streaming decoder v6, decode only	8.19 GB	56 s	flat
exp-126-stream-1	DiT streaming, 8 steps (A/B round 1)	3.44 GB	180 s	flat
exp-127-composed	composed exact pipeline, v1	8.57 GB	1266 s	flat
exp-130-stock	stock DiT + streamed encoder, 50 steps (true-stock latents)	5.87 GB	1136 s	flat
exp-131-ctxref-stream	streamed DiT on the frozen reference's path, 50 steps	3.49 GB	1151 s	flat
exp-132-stock-decode	stock tiled decode of the true-stock latents	16.92 GB	113 s	grew
exp-ref-v2	exact pipeline, waves	8.52 GB	1226 s	flat
exp-ref-v3	exact pipeline, neon	8.54 GB	1231 s	flat
exp-134-a2	A2, fox	8.61 GB	771 s	flat
exp-a2-v2	A2, waves	8.67 GB	752 s	flat
exp-a2-v3	A2, neon	8.70 GB	747 s	flat
exp-a3d-v2	A3-d, waves	8.67 GB	874 s	flat
exp-a3d-v3	A3-d, neon	8.64 GB	1002 s	flat

Full working log: reports/phase4b.md. Every number in this PDF is read from runs/*/result.json and gate.json by reports/wan_report_data.py when the report is built.